HOTEL MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

LINGESH SAI M

(241401046)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –
LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

2026-2027

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “Hotel Management System” is the bonafide work of LINGESH SAI M (241401046), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K. R.Sowmia, Ph.D.,
HEAD OF THE DEPARTMENT
Department of Computer Science and Business System
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr. M. L. Alphin Ezhil Manuel
COURSE FACULTY
Assistant Professor (SG)
Department of Computer Science and Business System
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 20 October 2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, LINGESH SAI M (241401046), hereby declare that the project report titled “Hotel Management System”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 20 October 2026

SIGNATURE OF THE STUDENT
LINGESH SAI M
241401046

ACKNOWLEDGEMENT

I express my sincere gratitude to the management of Rajalakshmi Engineering College for providing the infrastructure and learning environment for this project.

I am grateful to the Principal of Rajalakshmi Engineering College for encouraging students to undertake practical, application-oriented learning.

I thank the Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I thank the course faculty for the guidance and feedback provided during the development of this application.

I acknowledge Zoho Corporation for the Zoho Creator platform and learning resources, and thank my family and friends for their encouragement.

LINGESH SAI M

ABSTRACT

The Hotel Management System is a low-code web application developed using Zoho Creator to organise the day-to-day information of a small or medium hotel. It brings room details, guest records, bookings, extra-service charges, billing, housekeeping tasks and guest feedback together within one workspace. The project was developed as an individual academic project for CB23F35 – Low-Code Application Development using Zoho Creator.

The application is built around seven forms: Room_Type, Room, Guest, Booking (with a Services subform), Housekeeping_Task, Feedback and Holiday_List. Deluge scripts reject invalid dates and double bookings, calculate the bill automatically from the number of nights, extra services, a holiday-season surcharge, a loyalty discount and GST, and process check-in and check-out. At check-out the room is marked Dirty, a housekeeping task is created and the guest's stay count and loyalty tier are updated. A scheduled workflow sends next-day check-in reminders and marks no-shows.

Public-holiday dates are retrieved from the Nager.Date public-holidays API using the Deluge invokeurl task, with a duplicate check so that repeated runs do not insert the same holiday again; these dates drive the holiday-pricing rule. Reports, an Hotel Dashboard with occupancy, revenue and status charts, and role-based permission profiles for the administrator, receptionist and housekeeping staff complete the application. The report documents the study, design, implementation, testing, deployment and results.

Keywords: Low-code development; Zoho Creator; hotel management; Deluge scripting; API integration; workflow automation; booking system

PROJECT AT A GLANCE

Attribute	Details
Project Title	Hotel Management System
Application Domain	Hospitality / Hotel front-office and housekeeping
Platform	Zoho Creator (web application)
Application Type	Low-code business application
Primary Users	Administrator; receptionist; housekeeping staff (optional guest portal)
Core Forms	Room_Type; Room; Guest; Booking (with Services subform); Housekeeping_Task; Feedback; Holiday_List
Reports / Views	All Rooms; Available Rooms; All Guests; All Bookings; Today's Check-ins; Housekeeping Tasks; Feedback; Holiday_List
Dashboard / Pages	Hotel Dashboard; API Holiday Demo page
Scripting	Deluge functions and form scripts for overlap check, bill calculation, check-in / check-out, holiday retrieval and scheduled reminders
External Integration	Nager.Date public-holidays API; Zoho Mail notifications
Unique Features	Holiday-aware pricing; loyalty tiers; automatic housekeeping task on check-out; automatic no-show marking
Deployment Status	Built in Zoho Creator; publishing through Environments described in Section 7.1
Testing Evidence	14 test cases defined in Table 6.1 covering validation, billing, check-out, API and roles
Student Name	LINGESH SAI M
Register Number	241401046
Academic Year	2026–2027

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION **1**

1.1 Background . . . 1

1.2 Problem Statement . . . 1

1.3 Objectives . . . 1

1.4 Scope of the Project . . . 2

1.5 Justification for a Low-Code Approach . . . 2

1.6 Organisation of the Report . . . 2

1.7 Development Methodology . . . 2

CHAPTER 2 – LITERATURE SURVEY AND SYSTEM STUDY **3**

2.1 Study of the Existing System . . . 3

2.2 Limitations of the Existing System . . . 3

2.3 Low-Code versus Traditional Development . . . 3

2.4 Literature Survey . . . 4

2.5 Emerging Trends in Low-Code Development . . . 4

2.6 Proposed System . . . 4

2.7 Feasibility Study . . . 5

2.8 Requirements Drawn from the Study . . . 5

CHAPTER 3 – REQUIREMENT ANALYSIS **6**

3.1 Stakeholders and User Roles . . . 6

3.2 Functional Requirements . . . 6

3.3 Non-Functional Requirements . . . 7

3.4 Hardware and Software Requirements . . . 7

3.5 Use Case Model . . . 7

3.6 Assumptions and Constraints . . . 8

3.7 Requirements Traceability . . . 8

CHAPTER 4 – SYSTEM DESIGN **9**

4.1 System Architecture . . . 9

4.2 Data Model . . . 9

4.3 Form Design . . . 10

4.4 Process and Workflow Design . . . 12

4.5 User Interface and Navigation Design . . . 12

4.6 Access Control Design . . . 13

4.7 Design Considerations . . . 13

CHAPTER 5 – IMPLEMENTATION **14**

5.1 Application Setup . . . 14

5.2 Forms and Data Capture . . . 14

5.3 Reports . . . 15

5.4 Pages and Dashboards . . . 15
5.5 Deluge Scripting . . . 16
5.6 Workflows and Automation . . . 17
5.7 Integrations . . . 17
5.8 User Management and Role-Based Access . . . 18
5.9 AI and Analytics Features . . . 18
5.10 Pro-Code Extension . . . 18
5.11 Mobile and Portal Access . . . 18

CHAPTER 6 – TESTING AND DEBUGGING 19

6.1 Testing Strategy . . . 19
6.2 Test Cases . . . 19
6.3 Debugging Deluge Scripts . . . 20
6.4 Defect Log . . . 20
6.5 User Acceptance Testing . . . 20

CHAPTER 7 – DEPLOYMENT, SECURITY AND GOVERNANCE 21

7.1 Environments and Deployment Strategy . . . 21
7.2 Performance Optimisation . . . 21
7.3 Security Features . . . 21
7.4 Governance and Administration . . . 22
7.5 Backup and Recovery . . . 22
7.6 Maintenance and Support . . . 22

CHAPTER 8 – RESULTS AND DISCUSSION 23

8.1 Application Output . . . 23
8.2 Achievement of Objectives . . . 23
8.3 Comparison with the Existing System . . . 24
8.4 Limitations of the Low-Code Approach . . . 24
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum . . . 24

CHAPTER 9 – CONCLUSION AND FUTURE ENHANCEMENTS 25

9.1 Conclusion . . . 25
9.2 Future Enhancements . . . 25
9.3 Learning Outcomes and Career Relevance . . . 25

REFERENCES 26

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 27

APPENDIX B – COURSE OUTCOME MAPPING 28

APPENDIX C – COMPLETE DELUGE SOURCE CODE 29

APPENDIX D – USER MANUAL 32

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 3

Table 2.2: Summary of Literature Survey 4

Table 3.1: Stakeholders and User Roles 6

Table 3.2: Functional Requirements 6

Table 3.3: Non-Functional Requirements 7

Table 3.4: Hardware and Software Requirements 7

Table 3.5: Use Case Description – Book a Room 7

Table 4.1: Entities and Relationships 9

Table 4.2: Field Specification – Room Form 10

Table 4.3: Field Specification – Guest Form 11

Table 4.4: Field Specification – Booking Form 11

Table 4.5: Role–Permission Matrix (A = Add, E = Edit, V = View, D = Delete) 13

Table 5.1: Reports Implemented 15

Table 5.2: Deluge Scripts Implemented 16

Table 5.3: Workflows Implemented 17

Table 5.4: Integrations Implemented 17

Table 6.1: Test Cases 19

Table 6.2: Defect Log 20

Table 6.3: User Acceptance Feedback 20

Table 7.1: Deployment Checklist 21

Table 7.2: Security Controls Implemented 22

Table 8.1: Mapping of Objectives to Outcomes 24

Table 8.2: Existing System versus Proposed Application 24

Table A.1: Feature Coverage Checklist 27

Table B.1: Mapping of Course Outcomes to Report Chapters 28

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 3

Figure 3.1: Use Case Diagram of the Hotel Management System 8

Figure 4.1: System Architecture of the Hotel Management System 9

Figure 4.2: Entity–Relationship Diagram of the Hotel Management System 10

Figure 4.3: Workflow / Process Flow for Booking, Check-in and Check-out 12

Figure 4.4: Navigation Structure of the Hotel Management System 13

Figure 5.1: Application Home and Navigation Sections 14

Figure 5.2: Booking Form with Services Subform in the Builder 14

Figure 5.3: Booking Form in Live Mode 14

Figure 5.4: All Bookings Report 15

Figure 5.5: Hotel Dashboard 15

Figure 5.6: Deluge Script for the Overlap Check (DS-01) 17

Figure 5.7: Workflow Configuration 17

Figure 5.8: Holiday Function Executed Successfully 18

Figure 5.9: Roles and Permission Profiles Configuration 18

Figure 6.1: Logs or Debug Output for a Script 20

Figure 7.1: Environment Publishing Status 21

Figure 7.2: Version History and Application Administration 22

Figure 8.1: Rooms Report 23

Figure 8.2: Booking Record with Calculated Bill 23

Figure 8.3: Housekeeping Task Created at Check-out 23

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
GST	Goods and Services Tax
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
PMS	Property Management System
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

A hotel runs on timing and information. Every day the front desk must know which rooms are free, which guests are arriving or leaving, which extra services have been consumed, how much each guest owes, and which rooms must be cleaned before the next arrival. When these tasks are handled in paper registers, spreadsheets and phone calls, the information becomes fragmented. Two reservations can be accepted for the same room, a laundry slip can be lost before the bill is prepared, and a cleaner may not learn that a room has been vacated until someone walks over and says so.

A structured application gives the staff one common place to keep room, guest and booking records and to retrieve them through reports. The Hotel Management System was created in Zoho Creator as a low-code solution to these record-management and coordination needs. It is an academic prototype, but it includes realistic business rules such as overlap prevention, automatic billing with tax, loyalty discounts and holiday-season pricing, so that the project demonstrates more than simple data entry.

1.2 Problem Statement

Small and medium hotels often lack an affordable single application that maintains the room inventory, prevents double bookings, calculates bills accurately and coordinates housekeeping. The absence of such a system leads to several recurring problems:

- Double bookings caused by missed or overlapping reservations.
- Billing errors, because nights, services, taxes and discounts are worked out by hand.
- Delays in room turnover, because housekeeping is informed verbally after check-out.
- Loss of guest history, so repeat guests are not recognised or rewarded.
- No quick view of occupancy and revenue for the manager.

1.3 Objectives

- To design a structured data model and forms for room types, rooms, guests, bookings, housekeeping tasks and feedback.
- To prevent invalid dates and double bookings using Deluge validation scripts.
- To calculate the bill automatically, including extra services, a holiday-season surcharge, a loyalty discount and GST.
- To automate the effects of check-in and check-out: room status update, housekeeping task creation and loyalty tier update.

- To retrieve public-holiday data from the Nager.Date API, prevent duplicate insertion, and use it for pricing.
- To send reminder e-mails and mark no-shows using a scheduled workflow.
- To provide reports and a dashboard, and to restrict access by role using permission profiles.

1.4 Scope of the Project

The project covers room type and room management, guest registration, booking with overlap validation, an extra-services subform, automatic billing, check-in and check-out processing, housekeeping tasks, guest feedback, public-holiday retrieval, scheduled reminders, reports, a dashboard and role-based permissions. The application is created directly in Zoho Creator and accessed through its web interface.

Online payment gateways, SMS or WhatsApp messaging, integration with travel-booking portals, a dedicated guest mobile application and multi-property support are outside the scope of this version and are listed as future enhancements in Chapter 9.

1.5 Justification for a Low-Code Approach

Zoho Creator was selected because it provides visual form building, reports, dashboards, navigation and workflow capabilities without requiring a full custom web stack. Deluge supplies form logic, record fetching and updating, e-mail and REST calls, while the platform supplies hosting, user management and administration. This combination suits a student project because the application can be built quickly, changed easily when requirements shift, and tested directly in the browser. The platform also imposes constraints, such as exact field link names and usage limits, which are discussed in Chapters 6 to 8.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

1.7 Development Methodology

The project followed a simple incremental life cycle. The existing manual process was studied first and turned into functional and non-functional requirements. The data model was then drawn and the forms were built one at a time, beginning with the master forms (Room_Type, Room and Guest) and ending with the transaction form (Booking). Deluge scripts were added after the forms they depend on, and each script was tested immediately with valid and invalid input before the next one was started.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In a typical manually run hotel, a guest enquires by phone or at the desk. The receptionist looks through a register or spreadsheet to find a free room, writes the booking and notes the guest's details on paper. During the stay, food, laundry and other services are written on slips. At check-out the receptionist adds up the room charge and the slips, applies tax and any discount by hand and prepares the bill. Housekeeping is told verbally that the room is vacant, and the manager prepares an occupancy and revenue summary from the register at the end of the month. The proposed application brings these records into Zoho Creator forms, scripts and reports.

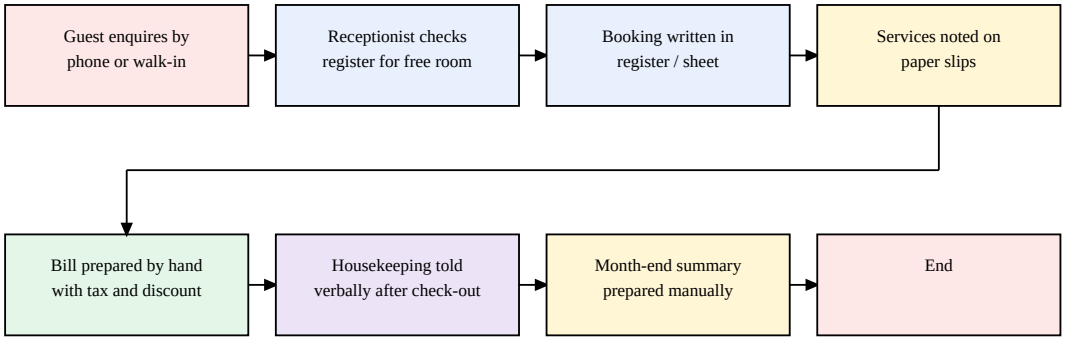


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- Room availability is not visible in real time, so double bookings are possible.
- Bills are calculated manually and discounts are applied inconsistently.
- Guest history is not retained in a searchable form, so repeat guests are not recognised.
- Housekeeping coordination depends on verbal messages, which delays room turnover.
- Occupancy and revenue summaries require repeated manual effort and are always out of date.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Usually longer for a custom hotel application	Visual configuration enables rapid prototyping
Skills required	Programming, framework and deployment skills	Platform knowledge and Deluge for custom logic
Hosting and infrastructure	Must be selected and maintained	Hosting provided by Zoho Creator
Mobile application	Needs responsive or native implementation	Creator supports mobile access

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Security and access control	Custom implementation and testing	Platform permission profiles and roles
Maintenance and change	Code changes and redeployment	Visual edits and workflow changes
Customisation limits	High flexibility with sufficient effort	Bound by platform capabilities and Deluge rules
Cost	Depends on hosting and engineering effort	Depends on Zoho plan or trial and usage limits

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source	Title / Product	Key Findings	Relevance to this Project
1	Zoho Corporation	Zoho Creator Help Documentation	Documents forms, reports, pages, workflows and application settings.	Primary platform reference.
2	Zoho Corporation	Deluge Script Reference	Documents Deluge syntax, record fetch and insert, sendmail and invokeurl.	Basis of all scripts in Appendix C.
3	Nager.Date	Public Holiday API	Provides public holiday data by country and year.	Source for the holiday-pricing rule.
4	Bardi, J. A.	Hotel Front Office Management, Wiley	Describes reservations, registration, room status, billing and check-out in hotels.	Source of the front-office process modelled in this project.
5	Gartner	Magic Quadrant for Enterprise Low-Code Application Platforms	Industry analysis of low-code platforms and enterprise use.	Background for the low-code approach.

2.5 Emerging Trends in Low-Code Development

Relevant trends include visual application development, citizen development, workflow automation, API-based integration and dashboard-driven reporting. In hospitality, property management systems are moving to the cloud, with automated reminders, integrated billing and housekeeping coordination, and pricing that responds to demand and calendar events. This project applies a small and transparent version of these ideas: holiday-aware pricing driven by a public API and automated workflows built entirely with Zoho Creator’s visual tools and Deluge.

2.6 Proposed System

The proposed Hotel Management System keeps room, guest, booking and housekeeping data in a single Zoho Creator application. Reports and navigation views support record review, and the Hotel Dashboard gives managers a quick view of occupancy and revenue. In addition to the functions every hotel needs, the system includes four features intended to set it apart from a basic booking form while remaining practical to build:

- **Holiday-aware pricing.** The nightly rate is raised by a configurable percentage for every night that falls on a public holiday retrieved from the Nager.Date API.
- **Loyalty tiers.** A guest’s completed-stay count drives automatic Silver and Gold tiers, each with a discount applied to later bills.

- **Check-out automation.** Checking a guest out marks the room Dirty and creates a housekeeping task, and completing that task returns the room to Available.
- **Scheduled automation.** A daily workflow e-mails guests arriving the next day and marks overdue confirmed bookings as No Show.

2.7 Feasibility Study

Technical feasibility. Every function of the proposed system can be built with standard Zoho Creator features: forms, lookups, a subform, reports, a dashboard page, form workflows, a schedule and Deluge scripts. The only external dependency is a public REST API that needs no key, and the user needs nothing more than a web browser.

Economic feasibility. No hardware or server has to be purchased, because the application is hosted by Zoho Creator. The cost is limited to the Zoho plan or trial used for development, which is small compared with the cost of building and hosting a custom application of the same scope.

Operational feasibility. The screens use plain labels and dropdowns, and each role sees only the menu it needs, so front-desk and housekeeping staff can be trained in a short session. Automatic billing and automatic housekeeping tasks replace work that staff already do by hand, so the system fits into the existing routine of the hotel.

2.8 Requirements Drawn from the Study

Each limitation found in the manual system was turned into a requirement for the proposed application, so that every feature has a reason to exist:

- Invisible availability and double bookings led to the overlap validation (FR-05).
- Manual and inconsistent billing led to the automatic bill with holiday surcharge, loyalty discount and GST (FR-07).
- Lost service slips led to the Services subform inside the booking (FR-06).
- Verbal housekeeping coordination led to the automatic task at check-out and the Room Ready workflow (FR-09, FR-10).
- Unrecognised repeat guests led to the stay count and loyalty tiers (FR-11).
- Manual month-end summaries led to the reports and the Hotel Dashboard (FR-14).

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator / App Owner	Manages the hotel application	Configure room types, rooms and rates; manage users and permissions; run the holiday fetch; view the dashboard and logs
Receptionist	Front-desk staff	Register guests; create and edit bookings; process check-in and check-out; add services; record feedback
Housekeeping Staff	Room cleaning staff	View pending housekeeping tasks and mark them done
Guest (optional)	Hotel customer	View own bookings and submit feedback where portal access is enabled
Reviewer / Examiner	Evaluates the project	Review screenshots, test records and the feature checklist

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall maintain room types with base rate and capacity.	Administrator	High
FR-02	The system shall maintain rooms with number, type, floor and status.	Administrator	High
FR-03	The system shall register guests with contact and ID proof details.	Receptionist	High
FR-04	The system shall create bookings with guest, room, check-in and check-out dates.	Receptionist	High
FR-05	The system shall reject invalid dates and bookings that overlap another booking of the same room.	System	High
FR-06	The system shall allow extra services to be added to a booking through a subform.	Receptionist	Medium
FR-07	The system shall calculate nights, charges, holiday surcharge, discount, GST and total automatically.	System	High
FR-08	The system shall retrieve public holidays from an external API and skip records already stored.	Administrator	High
FR-09	The system shall set the room to Occupied on check-in and Dirty on check-out.	System	High
FR-10	The system shall create a housekeeping task automatically on check-out.	System	High
FR-11	The system shall update the guest stay count and loyalty tier after check-out.	System	Medium
FR-12	The system shall e-mail next-day arrival reminders and mark no-shows through a scheduled workflow.	System	Medium
FR-13	The system shall capture guest feedback with a rating.	Receptionist / Guest	Low
FR-14	The system shall display a dashboard with occupancy, revenue and booking statistics.	Administrator / Receptionist	Medium
FR-15	The system shall restrict access to forms and reports by role.	Administrator	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Usability	Forms and navigation should use clear labels, and dropdowns should replace free text wherever possible.
NFR-02	Reliability	API errors and validation failures should be reported to the user and visible in the Logs.
NFR-03	Security	Only authorised roles should access or modify records; guest ID details should be restricted.
NFR-04	Maintainability	Forms, reports and Deluge scripts should have clear names, comments and documented purposes.
NFR-05	Integration	API requests should handle response data and avoid duplicate holiday records.
NFR-06	Accuracy	Bill amounts should match a manual calculation for the same booking.
NFR-07	Availability	Application access depends on the Zoho Creator service and successful publishing.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator web application
Scripting language	Deluge
External service	Nager.Date public-holidays API (no key required)
Notification service	Zoho Mail through the Deluge sendmail task
Browser	Modern web browser such as Chrome
Development device	Computer with internet access
Data storage	Zoho Creator application records

3.5 Use Case Model

The administrator configures rooms and rates, runs the holiday fetch and reviews the dashboard. The receptionist registers guests, creates bookings and processes arrivals and departures. Housekeeping staff work only with housekeeping tasks. The guest can submit feedback where portal access is enabled.

Table 3.5: Use Case Description – Book a Room

Item	Description
Use Case ID	UC-03
Use Case Name	Book a room
Actor(s)	Receptionist
Preconditions	The guest exists in the Guest form; room types and rooms are configured
Main Flow	Open the Booking form; select the guest and room; enter the check-in and check-out dates; add services; submit. The system validates the dates and availability, calculates the bill and saves the booking with status Confirmed.
Alternate Flow	If the dates are invalid or the room is already booked for those dates, an alert is shown and the record is not saved.
Postconditions	The booking is stored with nights, charges and total amount calculated

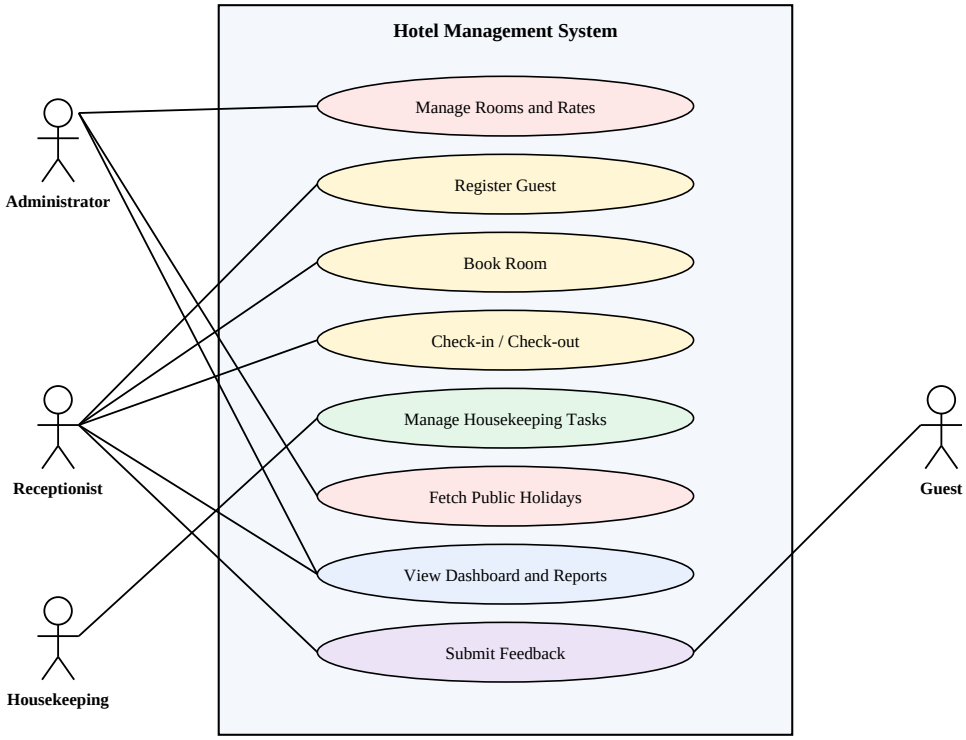


Figure 3.1: Use Case Diagram of the Hotel Management System

3.6 Assumptions and Constraints

- The hotel has a single property, and each room belongs to exactly one room type with one base rate.
- The holiday surcharge is a fixed percentage of the nightly rate and applies only to public holidays returned by the Nager.Date API for the chosen country and year.
- The GST percentage used in the bill is a configured value and must be updated if the tax slab changes.
- Check-in and check-out are stored by date only, and payments are recorded as Pending or Paid with no payment gateway.

3.7 Requirements Traceability

Each important requirement is traced to the part of the application that implements it and to the test case that checks it, so that nothing is built without a purpose and nothing is left untested:

- FR-05 (overlap and date validation) is implemented by DS-01 and checked by TC-03, TC-04 and TC-05.
- FR-07 (automatic bill) is implemented by DS-02 and checked by TC-06 to TC-09.
- FR-09, FR-10 and FR-11 (check-in, check-out, housekeeping task and loyalty tier) are implemented by DS-03 and checked by TC-10 and TC-11.
- FR-08, FR-12 and FR-15 are implemented by DS-04, DS-05 and the permission profiles, and checked by TC-12, TC-13 and TC-14.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

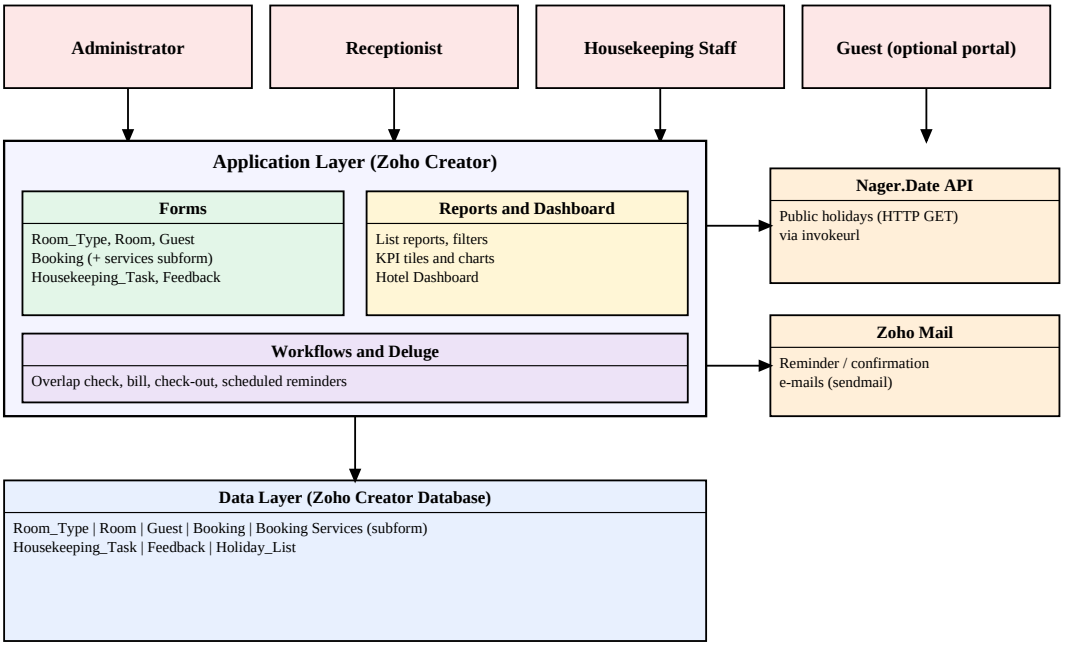


Figure 4.1: System Architecture of the Hotel Management System

Users access the application through the Zoho Creator web interface. The application layer contains the forms for rooms, guests, bookings, housekeeping and feedback, the reports, the dashboard and the workflows written in Deluge. Zoho Creator manages persistence and hosting. Holiday data is obtained from the external Nager.Date API through invokeurl and stored in the Holiday_List form, while reminder e-mails are sent through Zoho Mail using the sendmail task.

4.2 Data Model

A Room_Type has many Rooms, and a Room and a Guest each have many Bookings. The Booking form is the central transaction form. Its Services subform stores the extra services consumed during the stay. When a booking is checked out, a Housekeeping_Task is created for its room, and each completed stay can have one Feedback record. Holiday_List is not linked by a lookup; the billing script reads it by date when it calculates the holiday surcharge.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Room_Type	Name, Base_Rate, Capacity	Room	One type has many rooms	Lookup field in Room
Room	Room_Number, Room_Type, Floor, Room_Status	Booking; Housekeeping_Task	One room has many bookings	Lookup field in Booking

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Guest	Guest_Name, Phone, Email, ID, Stay_Count, Loyalty_Tier	Booking	One guest has many bookings	Lookup field in Booking
Booking	Guest, Room, dates, Status, charges, Total	Guest; Room; Services; Feedback	Central transaction form	Lookups and subform
Booking Services	Service_Name, Quantity, Unit_Price	Booking	Many lines per booking	Subform
Housekeeping_Task	Room, Booking_Ref, Task_Status	Room; Booking	Created at check-out	Deluge insert
Feedback	Booking, Rating, Comments	Booking	One per stay	Lookup field
Holiday_List	Holiday_Name, Holiday_Date	Booking (pricing rule)	Read by date	API and Deluge

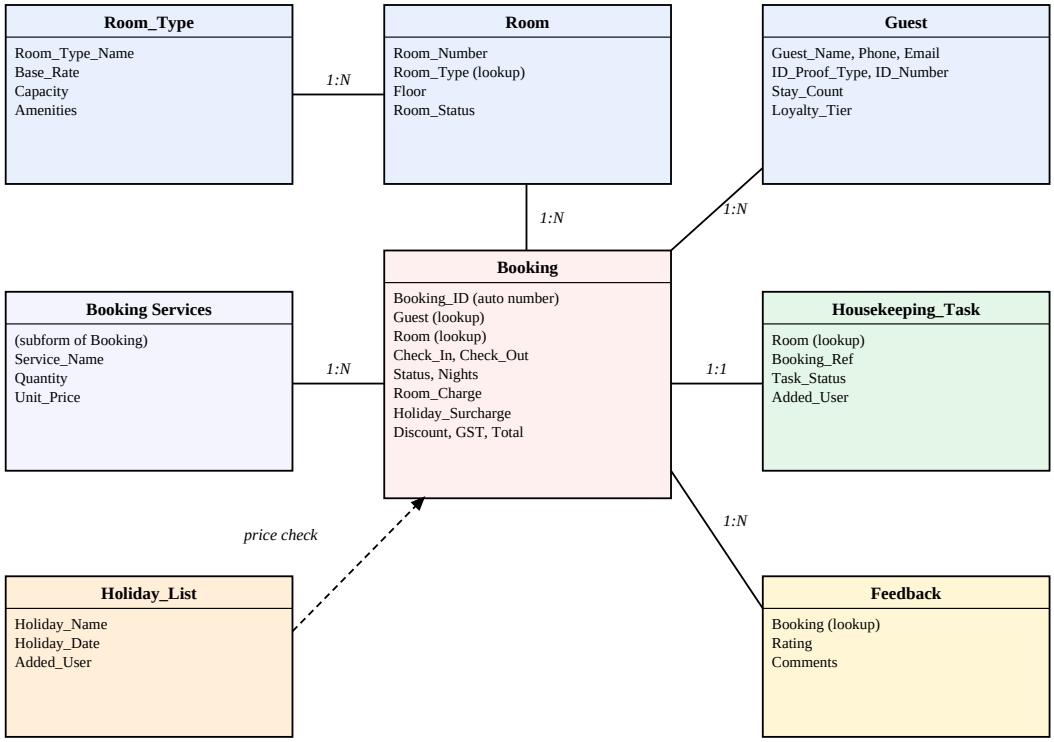


Figure 4.2: Entity–Relationship Diagram of the Hotel Management System

4.3 Form Design

The field specifications of the three main forms are given below. Link names are the names used in the Deluge scripts of Appendix C.

Table 4.2: Field Specification – Room Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Room Number	Room_Number	Single Line	Yes	Unique values only	Identifies the room
Room Type	Room_Type	Lookup (Room_Type)	Yes	Select an existing type	Gives the base rate
Floor	Floor	Number	Yes	Whole number	—
Room Status	Room_Status	Dropdown	Yes	Default Available	Available, Occupied, Dirty, Maintenance

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Notes	Notes	Multi Line	No	—	Optional remarks

Table 4.3: Field Specification – Guest Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Guest Name	Guest_Name	Single Line	Yes	—	Full name
Phone	Phone	Phone	Yes	Valid number format	Contact number
Email	Email	Email	Yes	Valid e-mail format	Used for reminders
ID Proof Type	ID_Proof_Type	Dropdown	Yes	Aadhaar, Passport, Driving Licence, Voter ID	Check-in requirement
ID Number	ID_Number	Single Line	Yes	—	Restricted access
Stay Count	Stay_Count	Number	No	Default 0	Updated by script
Loyalty Tier	Loyalty_Tier	Dropdown	No	Default None; Silver, Gold	Updated by script

Table 4.4: Field Specification – Booking Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Booking ID	Booking_ID	Auto Number	Auto	Prefix BK-	Unique reference
Guest	Guest	Lookup (Guest)	Yes	—	Who is staying
Room	Room	Lookup (Room)	Yes	—	Which room
Check In / Check Out	Check_In, Check_Out	Date	Yes	Check-out after check-in	Validated by DS-01
Adults	Adults	Number	Yes	Minimum 1	—
Early Check-in	Early_Check_In	Checkbox	No	—	Request flag
Status	Status	Dropdown	Yes	Confirmed, Checked In, Checked Out, Cancelled, No Show	Drives room status
Services	Services	Subform	No	Service_Name (Dropdown), Quantity (Number), Unit_Price (Decimal)	Extra services
Nights, Room Charge, Holiday Surcharge, Service Total, Discount, GST Amount, Total Amount	Nights, Room_Charge, Holiday_Surcharge, Service_Total, Discount, GST_Amount, Total_Amount	Number / Currency	Auto	Calculated by DS-02	Read-only
Payment Status	Payment_Status	Radio	Yes	Pending, Paid	Default Pending

Taken together, the forms use well over eight distinct field types: Single Line, Multi Line, Email, Phone, Number, Decimal, Currency, Date, Dropdown, Radio, Checkbox, Auto Number, Lookup and Subform.

4.4 Process and Workflow Design

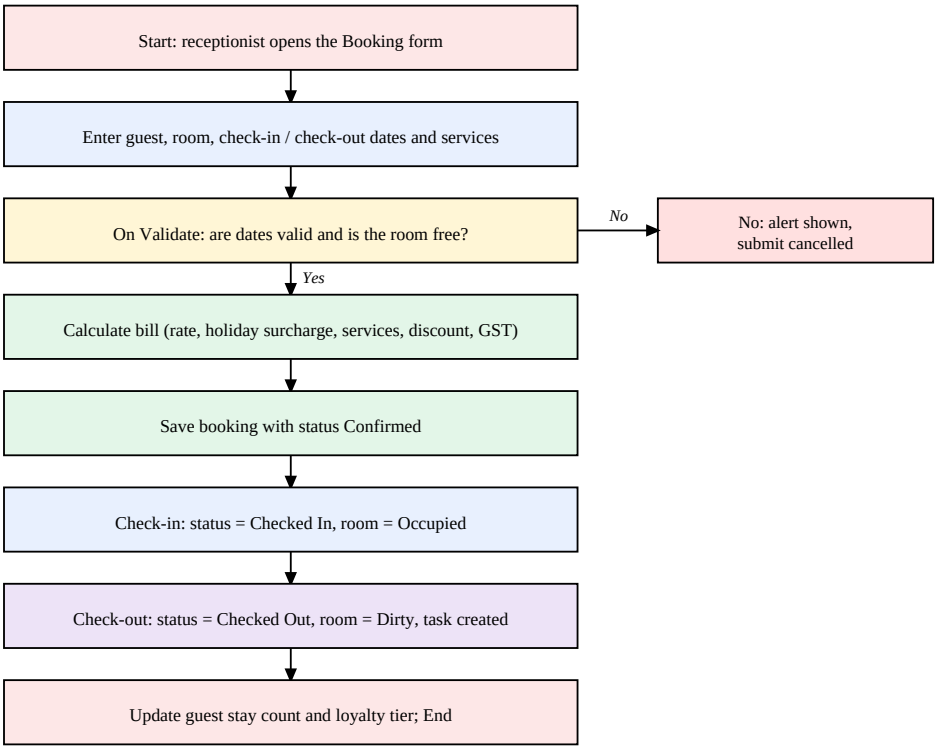


Figure 4.3: Workflow / Process Flow for Booking, Check-in and Check-out

A booking starts when the receptionist submits the Booking form. On validation, the first script rejects a check-out date that is not after the check-in date and any booking that overlaps an active booking of the same room. If the dates are acceptable, the second script calculates the nights, room charge, holiday surcharge, service total, loyalty discount, GST and final total, and the record is saved as Confirmed. When the receptionist later changes the status to Checked In, the room becomes Occupied. Changing it to Checked Out marks the room Dirty, creates a housekeeping task and increases the guest's stay count, promoting the guest to Silver or Gold when the thresholds are reached.

4.5 User Interface and Navigation Design

After login every user lands on the dashboard and sees only the menu sections permitted to their role. The administrator sees all sections. The receptionist sees guests, bookings, feedback and the dashboard, and housekeeping staff see only the housekeeping task report. Forms use grouped, clearly labelled fields, dropdowns for fixed choices and calculated read-only fields for billing values.

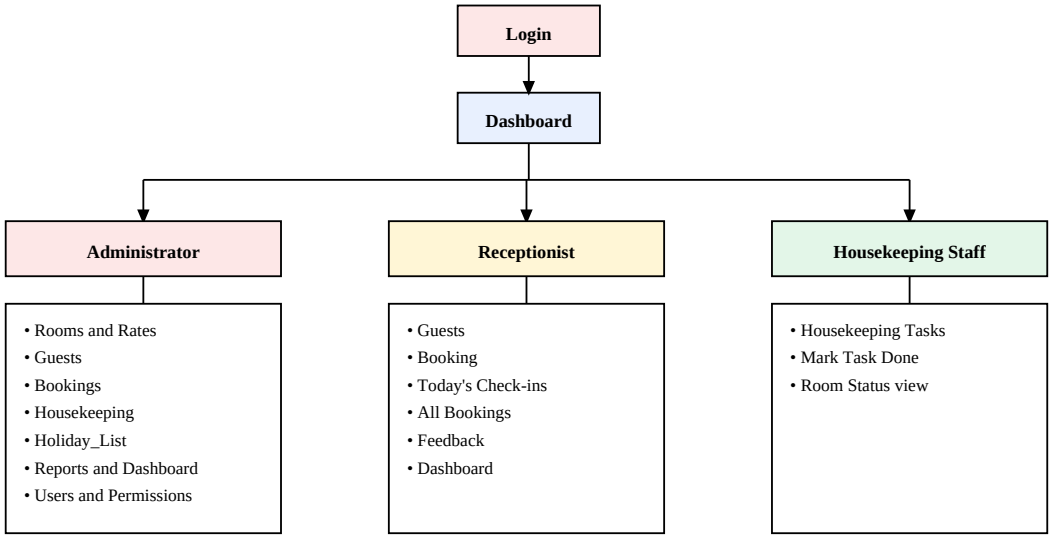


Figure 4.4: Navigation Structure of the Hotel Management System

4.6 Access Control Design

Table 4.5: Role–Permission Matrix (A = Add, E = Edit, V = View, D = Delete)

Component	Administrator	Receptionist	Housekeeping	Guest (optional)
Room_Type	A E V D	V	V	—
Room	A E V D	V, E (status)	V, E (status)	—
Guest	A E V D	A E V	—	V (own)
Booking	A E V D	A E V	V	V (own)
Housekeeping_Task	A E V D	V	E V	—
Feedback	V D	V	—	A V (own)
Holiday_List	A E V D	V	—	—
Dashboard	V	V	—	—

4.7 Design Considerations

Several choices were made to keep the design simple and reliable. Master data (room types, rooms and guests) is stored once and referred to by lookup fields, so a rate or a phone number is changed in one place only. Billing values are stored in the booking record rather than recalculated each time it is viewed, so an issued bill stays the same even if rates change later. Room and booking states are kept in dropdown fields with a fixed set of values, which makes scripts and reports easy to write and prevents spelling variations. Finally, the scripts are split by purpose (validation, billing, status processing and scheduling) so that each one is short enough to read and test on its own.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

A new application named Hotel Management System was created in Zoho Creator. The left menu was organised into sections for the Dashboard, Rooms, Guests, Bookings, Housekeeping, Feedback and Holidays, so that each role reaches its work in one or two clicks.

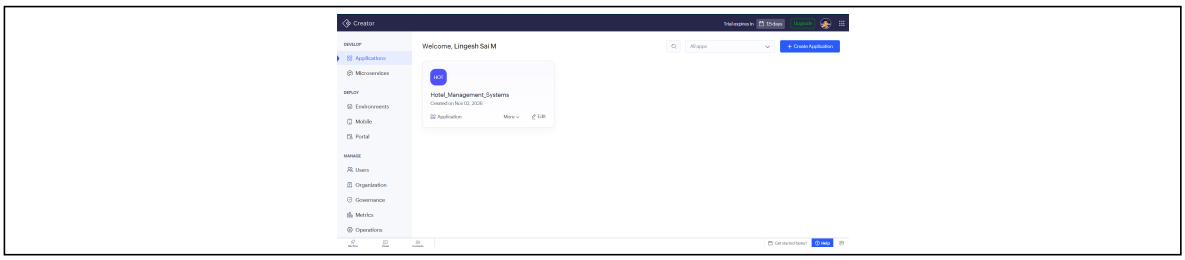


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Seven forms were created according to Tables 4.2 to 4.4. Required fields, dropdown lists and a unique constraint on Room_Number reduce invalid data at the point of entry. The Booking form contains the Services subform, in which the receptionist adds a row for each extra service, entering the service name, quantity and unit price. The billing fields are filled by script and cannot be edited by the user.

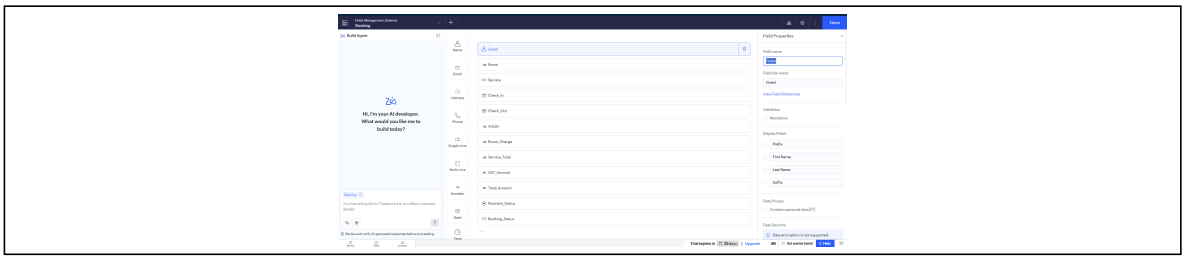


Figure 5.2: Booking Form with Services Subform in the Builder

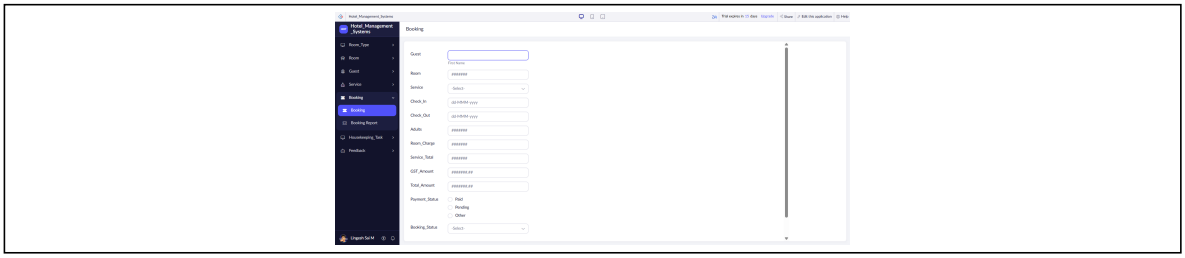


Figure 5.3: Booking Form in Live Mode

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Roles
All Rooms	List report	Room	Grouped by Room_Status	Administrator, Receptionist, Housekeeping
Available Rooms	List report	Room	Room_Status = Available	Administrator, Receptionist
All Guests	List report	Guest	Search by name or phone	Administrator, Receptionist
All Bookings	List report	Booking	Sorted by Check_In; filter by Status	Administrator, Receptionist
Today's Check-ins	List report	Booking	Check_In is today and Status is Confirmed	Administrator, Receptionist
Housekeeping Tasks	List report	Housekeeping_Task	Task_Status = Pending	Administrator, Housekeeping
Feedback	List report	Feedback	Sorted by Rating	Administrator, Receptionist
Holiday_List	List report	Holiday_List	Sorted by Holiday_Date	Administrator

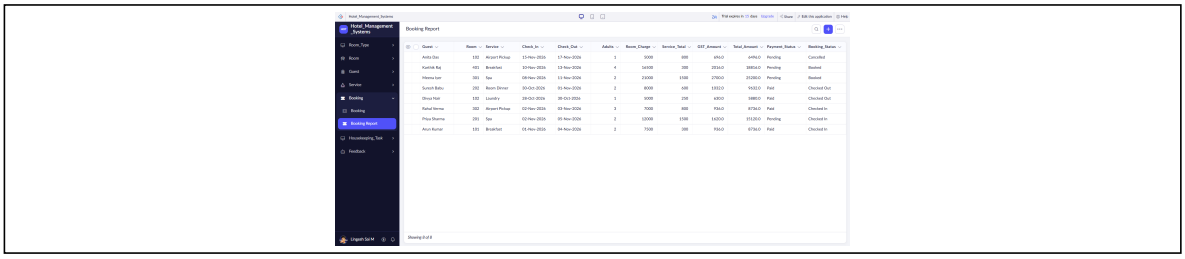


Figure 5.4: All Bookings Report

5.4 Pages and Dashboards

The Hotel Dashboard page was built with the UI builder. It shows KPI tiles for Total Rooms, Occupied Rooms, Occupancy Percentage, Today's Check-ins and Revenue, and three charts: Bookings by Room Type (bar), Booking Status Distribution (pie) and Revenue by Month (bar). A second page, API Holiday Demo, contains a button that runs the holiday-fetch function and displays its result message.

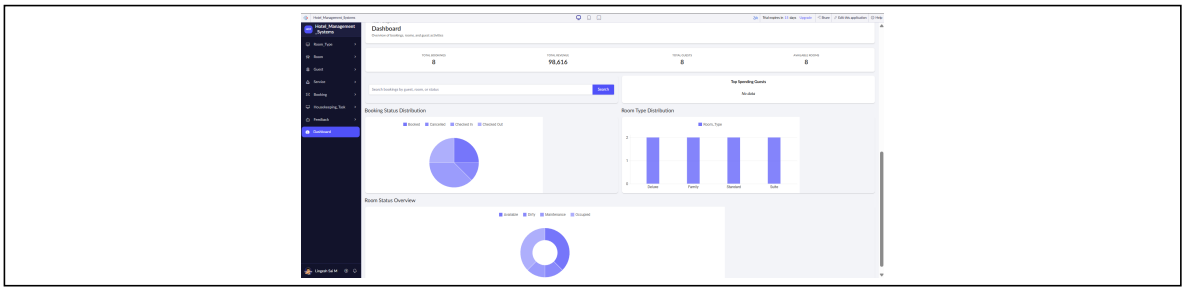


Figure 5.5: Hotel Dashboard

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Booking → On Validate	Reject invalid dates and overlapping bookings	Record fetch with criteria, count(), alert, cancel submit
DS-02	Booking → On Validate (after DS-01)	Calculate nights, surcharge, services, discount, GST and total	Variables, fetch, for each over subform, arithmetic, input assignment
DS-03	Booking → On Edit → On Success	Check-in and check-out effects, housekeeping task, loyalty tier	Conditions, record update, insert into
DS-04	Function fetch_holidays()	Import public holidays and skip existing records	invokeurl, toJSONList, for each, insert
DS-05	Scheduled workflow (daily)	Send reminders and mark no-shows	zoho.currentdate, addDay, sendmail, update
DS-06	Housekeeping_Task → On Edit → On Success	Return a cleaned room to Available	Fetch, update

DS-01 protects the room inventory. It compares the requested stay with every active booking of the chosen room and stops the submission if the periods overlap. Because the test is *existing check-in earlier than new check-out* and *existing check-out later than new check-in*, a new guest may arrive on the day the previous guest leaves.

```
// DS-01 | Form: Booking | Event: On Validate
if(input.Check_Out <= input.Check_In)
{
    alert "Check-out date must be after the check-in date.";
    cancel submit;
}
clash = Booking[Room == input.Room && Status != "Cancelled" && Status != "Checked Out"
                && Status != "No Show" && Check_In < input.Check_Out
                && Check_Out > input.Check_In];
if(clash.count() > 0)
{
    alert "This room is already booked for the selected dates.";
    cancel submit;
}
```

DS-02 calculates the bill. It reads the base rate from the room type, counts the nights, adds a surcharge for each stay night that matches a stored holiday, totals the subform rows, applies the loyalty discount and finally adds GST. The complete script is listed in Appendix C.2. The holiday-fetch function DS-04 sends a GET request to the Nager.Date endpoint, converts the response to a list of holidays and inserts only those whose date is not already in Holiday_List, so running it again does not create duplicates.

```
// DS-04 | Function: fetch_holidays() (extract)
response = invokeurl
[
    url :"https://date.nager.at/api/v3/PublicHolidays/2026/IN"
    type :GET
];
holidays = response.toJSONList();
for each holiday in holidays
{
    hDate = holiday.get("date").toDate();
    existing = Holiday_List[Holiday_Date == hDate];
    if(existing.count() == 0)
    {
```

```
        insert into Holiday_List
        [
            Holiday_Name = holiday.get("localName")
            Holiday_Date = hDate
            Added_User = zoho.loginuser
        ]
    }
}
```

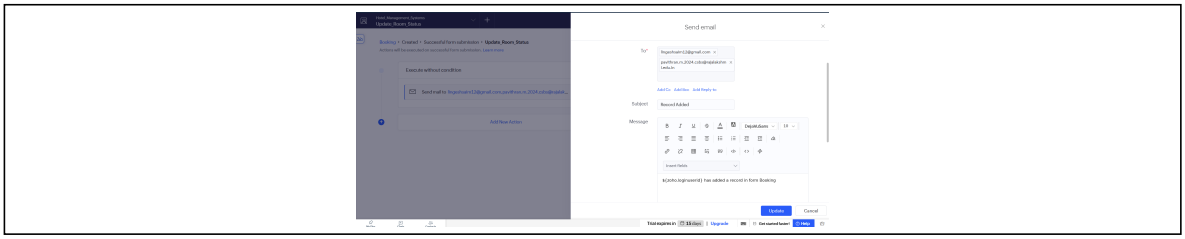


Figure 5.6: Deluge Script for the Overlap Check (DS-01)

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Booking validation and billing	Form workflow	Booking → On Create → On Validate	Always	Date check, overlap check, bill calculation	DS-01, DS-02
Check-in / Check-out	Form workflow	Booking → On Edit → On Success	Status changed	Update room, create task, update loyalty tier	DS-03
Fetch Holidays	Function / button	Click on API Holiday Demo page	Holiday not already stored	Insert new holiday records only	DS-04
Daily Reminder and No-show	Scheduled workflow	Daily at 8:00 AM	Check-in is tomorrow / date has passed	Send e-mail; set status No Show	DS-05
Room Ready	Form workflow	Housekeeping_Task → On Edit → On Success	Task_Status is Done	Set Room_Status to Available	DS-06

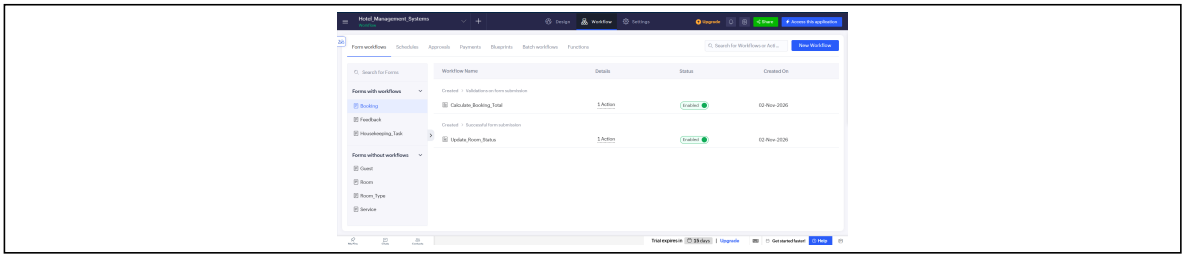


Figure 5.7: Workflow Configuration

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Public Holidays API	Nager.Date public-holidays endpoint	invokeurl GET	Public API; no key	Retrieve holiday records for Holiday_List
E-mail	Zoho Mail	sendmail task	Account sender	Reminders and booking notifications

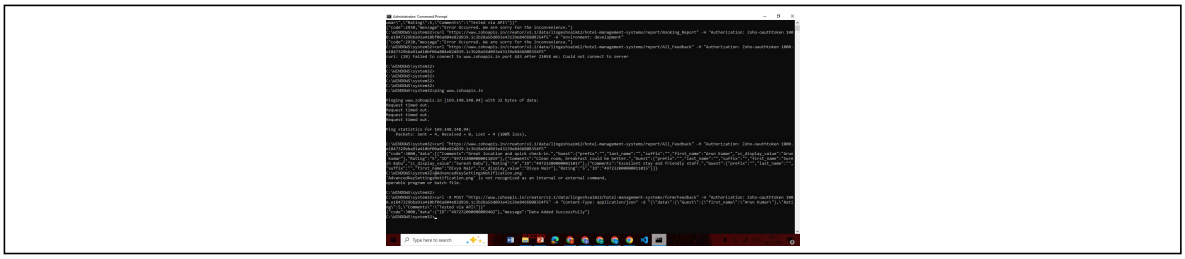


Figure 5.8: Holiday Function Executed Successfully

5.8 User Management and Role-Based Access

Permission profiles were created for the administrator, the receptionist and housekeeping staff, following the matrix in Table 4.5. Users were invited to the application and assigned to a profile in the Settings area. The housekeeping profile can see only the Housekeeping_Task and Room components, and guest identification fields are restricted to the administrator and the receptionist.

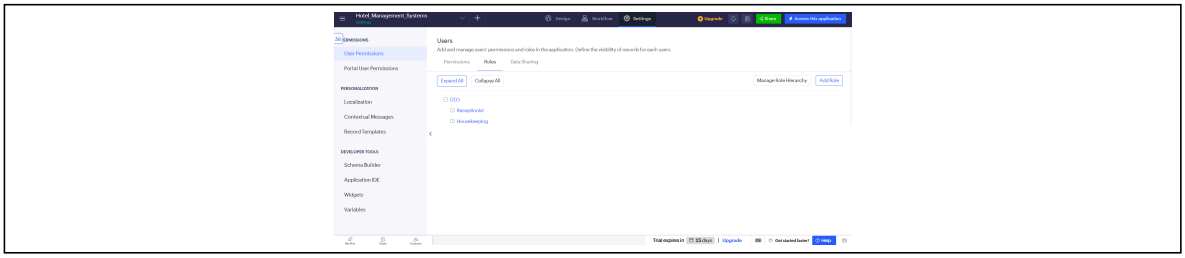


Figure 5.9: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

The dashboard charts provide the application's analytics: occupancy, booking status and revenue by month. Zia and the AI application builder may assist with generating forms or workflow drafts, but every form, script and workflow was reviewed and adjusted manually. No predictive model or OCR feature is claimed as implemented.

5.10 Pro-Code Extension

No separate pro-code extension using Zoho Catalyst, a custom JavaScript widget or a standalone REST service is part of this build. If pricing rules later become too complex for Deluge, the calculation could be moved into an external function called from the Booking form.

5.11 Mobile and Portal Access

The application is designed for use through the Zoho Creator web interface, and Creator's own mobile access can open the same application on a phone. A dedicated guest-facing portal for online booking is not part of this version and is proposed as a future enhancement.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was performed incrementally while building. Each form was first opened in live mode and filled with sample data. The business rules were then tested with normal, boundary and invalid input: a valid booking, a check-out earlier than check-in, an overlapping stay, and a stay that begins on the day the previous guest leaves. Bills were compared with a manual calculation, the holiday function was executed twice to confirm duplicate prevention, and each role was used to log in to confirm what it could see.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-03	Required fields on Guest form	Submit the empty form	Required-field messages appear		
TC-02	FR-04	Valid booking	Guest, free room, check-out after check-in	Record saved with status Confirmed		
TC-03	FR-05	Check-out before check-in	Enter an earlier check-out	Alert shown; submit cancelled		
TC-04	FR-05	Overlapping booking	Book a room for dates overlapping an existing booking	Alert shown; submit cancelled		
TC-05	FR-05	Back-to-back booking	Check-in on the previous guest’s check-out date	Booking accepted		
TC-06	FR-07	Bill without services or holiday	Three nights, no services	Total equals manual calculation		
TC-07	FR-06, 07	Bill with services	Add two subform rows	Service total added to the bill		
TC-08	FR-07	Holiday surcharge	Stay covering a stored holiday	Surcharge added for holiday nights only		
TC-09	FR-07, 11	Loyalty discount	Booking for a Silver or Gold guest	Discount applied at the tier rate		
TC-10	FR-09	Check-in	Set status to Checked In	Room becomes Occupied		
TC-11	FR-09, 10, 11	Check-out	Set status to Checked Out	Room Dirty; task created; stay count increases		
TC-12	FR-08	Holiday fetch, repeat run	Run the function twice	First run inserts; second adds none		
TC-13	FR-12	Scheduled reminder	Confirmed booking with check-in tomorrow	Reminder e-mail is sent		
TC-14	FR-15	Role access	Log in as housekeeping user	Only permitted components are visible		

6.3 Debugging Deluge Scripts

Scripts were debugged with *info* statements that print intermediate values, such as the number of overlapping records or the calculated nights, and with the Zoho Creator Logs, which show the form, the event and the error message when a script fails. The most common causes of errors in this kind of application are a field link name that differs from the one typed in the script, a comparison between a date and a date-time value, and a numeric field that is empty. Checking these three items resolved most problems encountered during development.



Figure 6.1: Logs or Debug Output for a Script

6.4 Defect Log

Defects found during testing are recorded in Table 6.2 with their root cause and fix.

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG01				
BUG02				
BUG03				

6.5 User Acceptance Testing

The application was demonstrated to a user in each role so that the receptionist, housekeeping and administrator workflows could be tried with realistic data. Their feedback is recorded below.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
	Receptionist		
	Housekeeping staff		
	Administrator		

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator Environments allow an application to be developed, tested and then published to production in stages. The strategy for this project is to build and test in the Development environment, move the application to Stage for acceptance testing with sample data, remove the test data, and finally publish to Production. Because publishing depends on the platform, the result of each publishing step is checked on the Environments page rather than assumed.

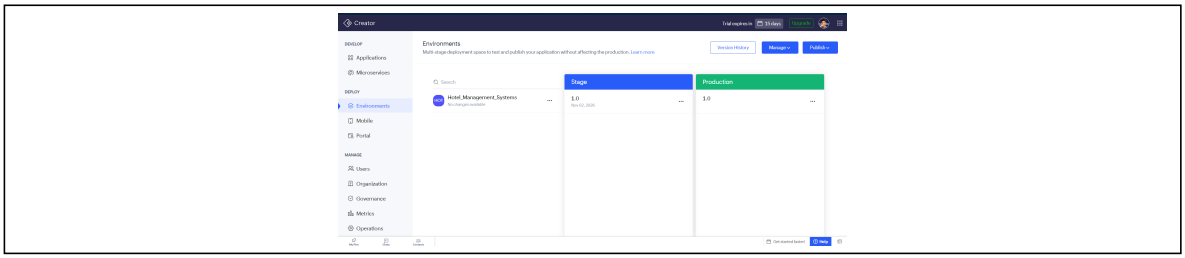


Figure 7.1: Environment Publishing Status

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done
1	All test cases executed in Development / Stage	
2	Sample and test data removed or isolated	
3	Form and workflow errors resolved	
4	Permissions checked for each role	
5	Holiday API tested with a repeat run	
6	Scheduled workflow tested	
7	Deployment / publish result verified	

7.2 Performance Optimisation

Three simple optimisations keep the application responsive. First, the billing script fetches the holiday records of the whole stay once, using a date-range criterion, instead of querying once for every night. Second, the holiday function checks for an existing record before inserting, which keeps Holiday_List small and free of duplicates. Third, reports such as Today’s Check-ins use criteria so that only the relevant records are loaded, instead of the whole Booking form. These improvements are structural; no timing figures were measured for this prototype.

7.3 Security Features

Table 7.2: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Permission profiles	All forms and reports	Unauthorised access or modification of records
Restricted ID fields	Guest form	Exposure of guest identity details
Required fields and unique Room_Number	Room, Guest and Booking forms	Missing or duplicate data
Date and overlap validation	Booking form	Invalid dates and double bookings
Account protection	Zoho account	Account misuse; two-factor authentication is recommended

7.4 Governance and Administration

The application is administered through the Zoho Creator account. Version History records the changes made to the application, Audit Logs record user activity, and Logs show script errors. Before final submission a backup or export of the application should be kept where the platform permits it, the application version should be noted, and access to the application should be verified from the owner's account.

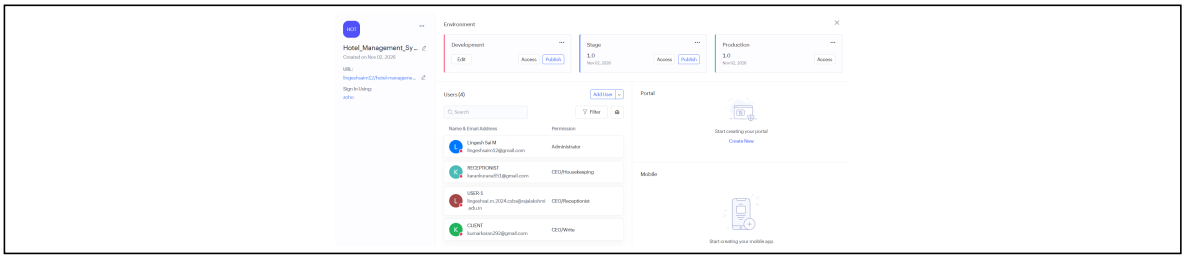


Figure 7.2: Version History and Application Administration

7.5 Backup and Recovery

Hotel data such as guest identity details and billing amounts must not be lost. A backup of the application data should therefore be taken regularly, and at least once before each publish to Production, using the backup or export option available to the account owner. If a publish or a script change damages the application, the previous version can be restored from Version History and the data can be re-imported from the latest backup. The Holiday_List form can always be rebuilt by running the holiday function again.

7.6 Maintenance and Support

After deployment the administrator is responsible for keeping the application accurate. Room rates and the GST percentage should be reviewed whenever the hotel changes its tariff, and the holiday function should be run once at the start of every year so that the pricing rule has current dates. Logs should be checked regularly for script errors, and any change to a form field name must be followed by a check of the Deluge scripts that refer to it. New staff are added by assigning them to one of the existing permission profiles, which avoids creating separate rules for each user.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

A representative scenario begins with the administrator creating the room types and rooms and loading the public holidays. The receptionist registers a guest and creates a booking. The system validates the dates, refuses a clash with another booking and fills in the nights, holiday surcharge, service total, discount, GST and total. On arrival the status is set to Checked In and the room shows as Occupied. On departure the status is set to Checked Out; the room becomes Dirty, a housekeeping task appears in the housekeeping report and the guest's stay count increases. When the cleaner marks the task done, the room returns to Available. The dashboard reflects each of these changes in its occupancy and status figures.

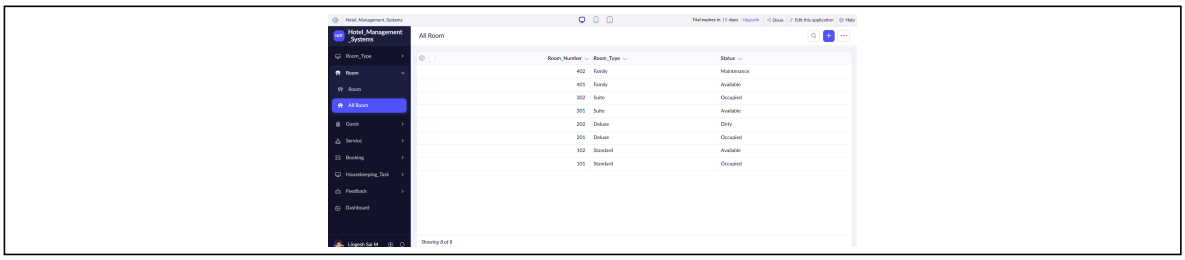


Figure 8.1: Rooms Report

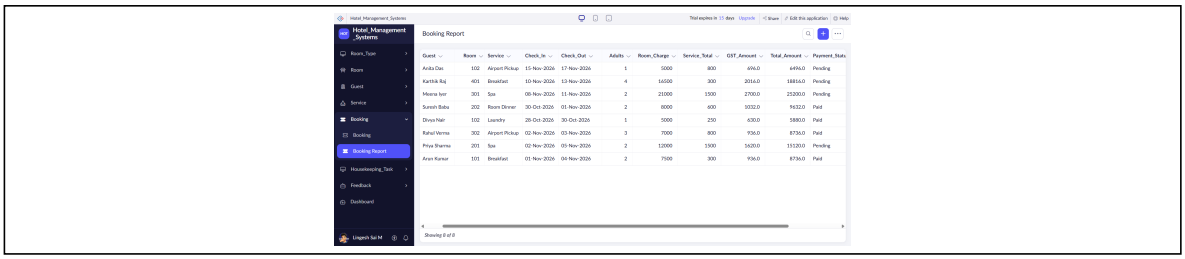


Figure 8.2: Booking Record with Calculated Bill

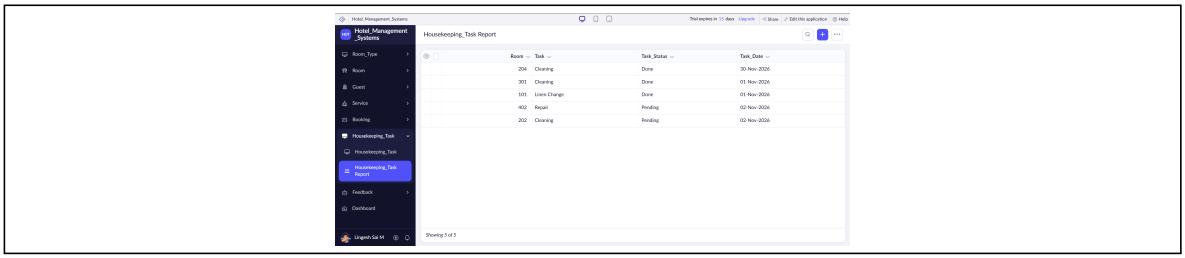


Figure 8.3: Housekeeping Task Created at Check-out

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome
Structured data model and forms	Seven forms (Chapter 4)	Room, guest, booking, housekeeping and feedback data held in linked forms
Prevent invalid dates and double bookings	DS-01	Overlapping and invalid bookings are rejected with an alert
Automatic bill	DS-02	Nights, surcharge, services, discount, GST and total are filled in on submission
Check-in and check-out automation	DS-03, DS-06	Room status, housekeeping task and loyalty tier follow the booking status
Holiday data without duplicates	DS-04	Holidays are imported once and reused for pricing
Reminders and no-shows	DS-05	Next-day arrivals are e-mailed and overdue bookings are marked No Show
Reports, dashboard and role control	Reports, dashboard, permission profiles	Each role sees its own menu, reports and dashboard tiles

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Availability check	Search register or sheet by hand	Automatic overlap validation	Double bookings are blocked
Billing	Manual calculation	Rule-based automatic calculation	Consistent totals and discounts
Housekeeping coordination	Verbal message	Task created at check-out	Faster room turnover
Guest recognition	Memory or notes	Stay count and loyalty tier	Repeat guests rewarded
Reporting	Manual month-end work	Dashboard and reports	Current figures at any time

8.4 Limitations of the Low-Code Approach

Low-code speeds up construction but does not remove the need for careful testing. Field link names must match exactly in every script, date and date-time values must not be mixed in comparisons, and complex pricing rules can become difficult to maintain in Deluge. Trial duration, usage limits and publishing behaviour of the platform can also affect delivery, and the holiday feature depends on the availability of an external API. These limitations were managed by testing each script separately and by keeping the pricing rules simple.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The application combines no-code configuration (forms, reports, the dashboard and navigation) with low-code scripting (Deluge for validation, billing, automation and the API call). No pro-code extension was required for this prototype. If a hotel later needs channel-manager integration or demand-based pricing, a pro-code service could be added alongside the Creator application. The project shows that low-code lowers the initial build effort but does not remove the need for testing, access control and troubleshooting.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Hotel Management System was developed in Zoho Creator to centralise room, guest, booking and housekeeping information and to automate billing and check-out processing. Besides the forms, reports and dashboard that every hotel application needs, it adds holiday-aware pricing driven by a public API, loyalty tiers, automatic housekeeping tasks and scheduled reminders. Together these features remove the main weaknesses of the manual system studied in Chapter 2: unseen room availability, manual billing and verbal housekeeping coordination.

The system is an academic prototype. Payment processing, messaging integrations and a guest-facing portal are not included, and the results of each test case and the outcome of publishing through Environments are recorded in Chapters 6 and 7.

9.2 Future Enhancements

- Online payment gateway and downloadable invoice generation.
- SMS or WhatsApp notifications for booking confirmation and reminders.
- A guest self-service portal for room search, booking and bill viewing.
- Zoho Analytics dashboards for deeper revenue and occupancy analysis.
- A Zoho Catalyst function for demand-based dynamic pricing.
- Multi-property support and integration with travel booking portals.

9.3 Learning Outcomes and Career Relevance

The project provided practical experience in low-code application construction, form and report design, lookups and subforms, Deluge scripting, scheduled workflows, external API calls, role-based access and environment management. It also showed the importance of testing actual outcomes rather than assuming that a saved configuration works. These skills are relevant to low-code development, business analysis, application support and workflow automation roles.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 2 November 2026).

[2] Zoho Corporation, “Deluge Script Reference.” [Online]. Available: https://www.zoho.com/deluge/help/ (Accessed: 2 November 2026).

[3] Nager.Date, “Public Holiday API.” [Online]. Available: https://date.nager.at (Accessed: 2 November 2026).

[4] J. A. Bardi, *Hotel Front Office Management*. Hoboken, NJ, USA: John Wiley & Sons.

[5] K. C. Laudon and J. P. Laudon, *Management Information Systems: Managing the Digital Firm*. Pearson.

[6] Gartner, “Magic Quadrant for Enterprise Low-Code Application Platforms.” Gartner research series.

[7] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*. New York, NY, USA: McGraw-Hill Education.

[8] I. Sommerville, *Software Engineering*. Pearson.

[9] C. J. Date, *An Introduction to Database Systems*. Pearson.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

This checklist records each Zoho Creator feature of the course as Y (implemented and evidenced) or N (not implemented). Features marked N are not claimed. Any entry must be changed if it does not match what is actually built in the application.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Where in the Application	Report Section	Figure / Code No.	Done
I	Application created and sections / menu organised	Hotel Management System menu	5.1	Fig 5.1	Y
II	Forms with 8 or more distinct field types	Room, Guest and Booking forms	4.3, 5.2	Tables 4.2–4.4	Y
II	Lookup field	Room_Type, Guest and Room in Booking	4.2	Fig 4.2	Y
II	Subform	Services subform in Booking	4.3	Fig 5.2	Y
II	Field validations	Date and overlap validation (DS-01)	5.5	C.1	Y
II	List report with filter and grouping	All Bookings; All Rooms	5.3	Table 5.1	Y
II	Chart report; page / dashboard with UI Builder	Hotel Dashboard; API Holiday Demo	5.4	Fig 5.5	Y
II	Deluge form events	DS-01, DS-02, DS-03	5.5	C.1–C.3	Y
II	Deluge function, loops, collections	fetch_holidays	5.5	C.4	Y
II	E-mail notification	Daily reminder (DS-05)	5.6	C.5	Y
II	Form workflow	Booking and housekeeping workflows	5.6	Table 5.3	Y
II	Scheduled workflow	Daily reminder and no-show	5.6	C.5	Y
II	Approval process, Blueprint, Kanban / Calendar / Map report	Not used	—	—	N
II	Testing and debugging evidence	Test cases and defect log	6	Tables 6.1, 6.2	Y
III	Connection / REST API call (invokeurl)	Nager.Date	5.7	C.4	Y
III	Users, roles and permission profiles	Administrator, Receptionist, Housekeeping	5.8	Fig 5.9	Y
III	Webhook or Creator REST API	Not used	—	—	N
III	Zia / AI feature	Optional assistance only	5.9	—	N
IV	Performance optimisation	Date-range fetch; duplicate check	7.2	—	Y
IV	Deployment across environments	Environments	7.1	Fig 7.1	Y
IV	Security controls	Permissions, validation	7.3	Table 7.2	Y
IV	Governance (audit trail, administration)	Version History, Logs	7.4	Fig 7.2	Y
V	Pro-code extension (Catalyst / widgets)	Not built	5.10	—	N
V	Low-code limitations and spectrum analysis	Chapter 8	8.4, 8.5	—	Y

APPENDIX B – COURSE OUTCOME MAPPING

The table shows where each Course Outcome of CB23F35 is evidenced in this report and application. Attainment is self-assessed on a scale of 1 (Low), 2 (Medium) and 3 (High); the examiner may revise it.

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1.5, Ch. 2, Ch. 4.1	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, Ch. 5, Ch. 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7, 5.8	3	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

Basis of Self-Assessment

- CO1 (3): the architecture, the low-code versus traditional comparison and the justification for Zoho Creator are covered in Chapters 1, 2 and 4.
- CO2 (3): seven forms, reports, a dashboard, form workflows and Deluge scripts are built.
- CO3 (3): the Nager.Date API call, e-mail notification and permission profiles are implemented.
- CO4 (2): environment publishing and governance are described; production use is limited to a prototype.
- CO5 (2): the low-code spectrum is analysed, but no pro-code extension was built.

The evidence for each outcome can be traced through the figure, table and section numbers listed in Appendix A and in the chapters named above.

APPENDIX C – COMPLETE DELUGE SOURCE CODE

This appendix lists the Deluge scripts used in the Hotel Management System, grouped by form or function. Each script carries the Script ID from Table 5.2, its location or trigger and its purpose.

C.1 DS-01 – Booking: Date and Overlap Validation

```
// Script ID : DS-01
// Location  : Booking > On Create > On Validate
// Purpose   : Reject invalid dates and double bookings
if(input.Check_Out <= input.Check_In)
{
    alert "Check-out date must be after the check-in date.";
    cancel submit;
}
clash = Booking[Room == input.Room && Status != "Cancelled" && Status != "Checked Out"
                && Status != "No Show" && Check_In < input.Check_Out
                && Check_Out > input.Check_In];
if(clash.count() > 0)
{
    alert "This room is already booked for the selected dates.";
    cancel submit;
}
```

C.2 DS-02 – Booking: Bill Calculation

```
// Script ID : DS-02
// Location  : Booking > On Create > On Validate (after DS-01)
// Purpose   : Calculate nights, holiday surcharge, services, discount, GST, total
roomRec = Room[ID == input.Room];
typeRec = Room_Type[ID == roomRec.Room_Type];
rate = typeRec.Base_Rate;

nights = input.Check_In.daysBetween(input.Check_Out).abs();
if(nights < 1)
{
    nights = 1;
}
roomCharge = nights * rate;

// Holiday surcharge: 20 percent of the nightly rate for each holiday night
holidayRate = 0.20;
hol = Holiday_List[Holiday_Date >= input.Check_In && Holiday_Date < input.Check_Out];
surcharge = hol.count() * rate * holidayRate;

// Extra services from the subform
serviceTotal = 0;
for each s in input.Services
{
    serviceTotal = serviceTotal + (s.Quantity * s.Unit_Price);
}

// Loyalty discount
guestRec = Guest[ID == input.Guest];
discountRate = 0;
if(guestRec.Loyalty_Tier == "Gold")
{
    discountRate = 0.10;
}
else if(guestRec.Loyalty_Tier == "Silver")
{
    discountRate = 0.05;
}
subtotal = roomCharge + surcharge + serviceTotal;
discount = subtotal * discountRate;

// GST rate: set according to the GST slab applicable to the room tariff
gstRate = 0.12;
gst = (subtotal - discount) * gstRate;
```

```
input.Nights = nights;
input.Room_Charge = roomCharge;
input.Holiday_Surcharge = surcharge;
input.Service_Total = serviceTotal;
input.Discount = discount;
input.GST_Amount = gst;
input.Total_Amount = subtotal - discount + gst;
```

C.3 DS-03 – Booking: Check-in and Check-out Processing

```
// Script ID : DS-03
// Location  : Booking > On Edit > On Success
// Purpose   : Update room, create housekeeping task, update loyalty tier
roomRec = Room[ID == input.Room];
if(input.Status == "Checked In")
{
    roomRec.Room_Status = "Occupied";
}
if(input.Status == "Checked Out")
{
    roomRec.Room_Status = "Dirty";
    insert into Housekeeping_Task
    [
        Room = input.Room
        Booking_Ref = input.ID
        Task_Status = "Pending"
        Added_User = zoho.loginuser
    ]
    guestRec = Guest[ID == input.Guest];
    newCount = ifnull(guestRec.Stay_Count, 0) + 1;
    guestRec.Stay_Count = newCount;
    if(newCount >= 6)
    {
        guestRec.Loyalty_Tier = "Gold";
    }
    else if(newCount >= 3)
    {
        guestRec.Loyalty_Tier = "Silver";
    }
}
```

C.4 DS-04 – Holiday Fetch Function (Nager.Date API)

```
// Script ID : DS-04
// Location  : Function fetch_holidays() (called from API Holiday Demo page)
// Purpose   : Fetch public holidays, skip stored ones, insert only new records
string fetch_holidays()
{
    response = invokeurl
    [
        url :"https://date.nager.at/api/v3/PublicHolidays/2026/IN"
        type :GET
    ];
    holidays = response.toJSONList();
    for each holiday in holidays
    {
        hDate = holiday.get("date").toDate();
        existing = Holiday_List[Holiday_Date == hDate];
        if(existing.count() == 0)
        {
            insert into Holiday_List
            [
                Holiday_Name = holiday.get("localName")
                Holiday_Date = hDate
                Added_User = zoho.loginuser
            ]
        }
    }
    return "Holiday records imported successfully.";
}
```

C.5 DS-05 – Scheduled Reminder and No-show Marking

```
// Script ID : DS-05
// Location  : Schedule (daily, 8:00 AM) on form Booking
// Purpose   : Remind guests arriving tomorrow; mark overdue bookings as No Show
tomorrow = zoho.currentdate.addDay(1).toDate();
today = zoho.currentdate.toDate();
arrivals = Booking[Check_In == tomorrow && Status == "Confirmed"];
for each b in arrivals
{
    g = Guest[ID == b.Guest];
    sendmail
    [
        from :zoho.adminuserid
        to :g.Email
        subject :"Reminder: your hotel check-in is tomorrow"
        message :"Dear " + g.Guest_Name + ", we look forward to welcoming you on " + tomorrow
    ]
}
overdue = Booking[Check_In < today && Status == "Confirmed"];
for each n in overdue
{
    n.Status = "No Show";
}
```

C.6 DS-06 – Housekeeping: Room Ready

```
// Script ID : DS-06
// Location  : Housekeeping_Task > On Edit > On Success
// Purpose   : When cleaning is done, make the room available again
if(input.Task_Status == "Done")
{
    r = Room[ID == input.Room];
    r.Room_Status = "Available";
}
```

Field link names in the scripts must match the link names created in the application.

APPENDIX D – USER MANUAL

This manual explains how a new user can operate the Hotel Management System without help. The application is a Zoho Creator web application.

D.1 Accessing the Application

1. Open a modern web browser (such as Chrome) and go to the Zoho Creator sign-in page.
2. Sign in with the Zoho account that the application owner has shared the application with.
3. On the Applications home page, click Hotel Management System.
4. Use the left navigation menu to move between Dashboard, Rooms, Guests, Bookings, Housekeeping, Feedback and Holidays.

D.2 Instructions for Receptionist

1. Open Guests → Guest, enter the guest details and ID proof, and click Submit.
2. Open Bookings → Booking. Select the guest and room, enter the check-in and check-out dates, add any services in the subform and click Submit. If the room is not free for those dates, an alert appears.
3. On arrival, open the booking in All Bookings and change Status to Checked In.
4. On departure, change Status to Checked Out. The room is marked Dirty automatically.
5. Check the Total Amount in the booking record and set Payment Status to Paid after payment.

D.3 Instructions for Housekeeping Staff

1. Open Housekeeping → Housekeeping Tasks to see the pending tasks.
2. After cleaning, edit the task and set Task_Status to Done. The room becomes Available.

D.4 Instructions for Administrator

- Add room types with their base rates, and then add rooms, under Rooms.
- Open the API Holiday Demo page and run the holiday fetch. Running it again does not create duplicates.
- Review the Hotel Dashboard for occupancy, revenue and booking status.
- Manage users and permission profiles in Settings, and check Logs and Version History when needed.

D.5 Troubleshooting

- **“This room is already booked”:** another active booking overlaps the dates; choose another room or dates.
- **Total amount is zero:** check that the room type has a Base_Rate and the dates are correct.
- **Room still shows Dirty:** the housekeeping task has not been marked Done.